

ChainTrack B2B Reverse Bidding Platform - Replit Deployment Prompt

Quick Start Command

Copy and paste this command into your Replit terminal to deploy ChainTrack:

Bash

```
# Clone the repository
git clone <GITHUB_REPO_URL> chaintrack && cd chaintrack

# Install dependencies
pnpm install

# Set up environment variables (create .env file with your credentials)
cat > .env << EOF
DATABASE_URL=mysql://user:password@host/dbname
JWT_SECRET=your-secret-key-here
VITE_APP_ID=your-app-id
OAUTH_SERVER_URL=https://api.manus.im
VITE_OAUTH_PORTAL_URL=https://portal.manus.im
OWNER_OPEN_ID=owner-id
OWNER_NAME=Owner Name
VITE_APP_TITLE=ChainTrack
VITE_APP_LOGO=https://your-logo-url.png
BUILT_IN_FORGE_API_URL=https://api.manus.im
BUILT_IN_FORGE_API_KEY=your-api-key
EOF

# Push database schema
pnpm db:push

# Start the development server
pnpm dev
```

What is ChainTrack?

ChainTrack is a **B2B wholesale electronics reverse auction platform** that connects suppliers and buyers for trading refurbished phones and electronics.

Key Features:

For Suppliers:

- 🎯 **Real-time Reverse Auctions** - Bid on buyer-posted auctions
- 📊 **Dashboard** - Track active auctions and bidding statistics
- 📁 **Profile Management** - Manage company information and specialties
- 📈 **Performance Metrics** - View ratings and transaction history
- 🔔 **Bid Notifications** - Real-time updates on auction status

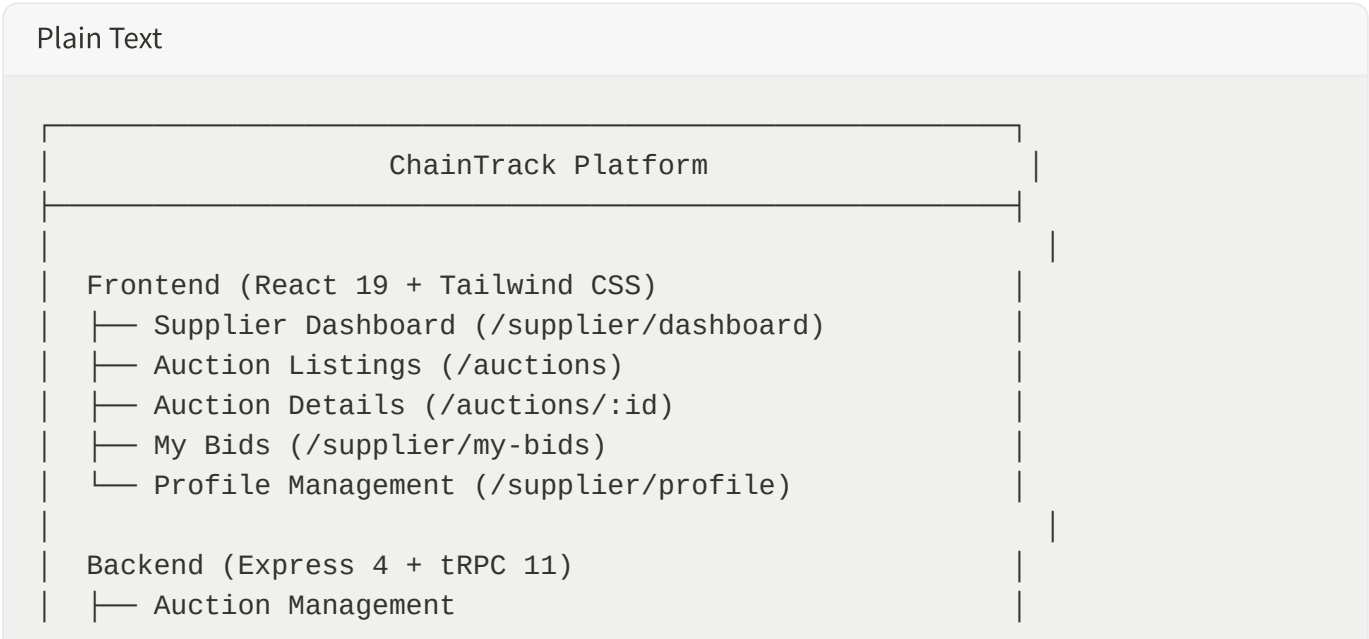
For Buyers:

- 📋 **Create Auctions** - Post reverse auction listings
- 🏆 **Competitive Bidding** - Get best prices from multiple suppliers
- ✅ **Supplier Verification** - View verified supplier ratings
- 🎥 **Live Inspections** - Request video inspections before purchase
- 💳 **Secure Payments** - Safe transaction processing

Platform Features:

- 🔑 **OAuth Authentication** - Secure user login
- 📱 **Responsive Design** - Works on desktop and mobile
- ⚡ **Real-time Updates** - Live bid tracking
- 🌐 **Global Reach** - Multi-currency support
- 📊 **Analytics** - Comprehensive platform metrics

Architecture Overview



```
├── Bidding Engine
├── Supplier Verification
├── Transaction Processing
├── User Authentication
|
Database (MySQL/TiDB)
├── Users & Roles
├── Auctions & Bids
├── Suppliers & Ratings
├── Transactions & Payments
├── Inspections & Verification
```

Database Schema

ChainTrack uses 9 core tables:

1. Users Table

Plain Text

```
id (Primary Key)
openId (OAuth identifier)
name, email
role (user/admin)
createdAt, updatedAt
```

2. Auctions Table

Plain Text

```
id (Primary Key)
buyerId (Foreign Key → users)
title, description
productType, quantity
condition, gradeRequired
startingPrice, reservePrice, currentLowestBid
startDate, endDate
deliveryLocation, paymentTerms
status (draft/active/closed/completed)
```

3. Bids Table

Plain Text

```
id (Primary Key)
auctionId (Foreign Key → auctions)
supplierId (Foreign Key → users)
bidPrice, quantity
paymentTerms, notes
estimatedDelivery
status (active/accepted/rejected/withdrawn)
```

4. Suppliers Table

Plain Text

```
id (Primary Key)
userId (Foreign Key → users)
companyName, contactPerson
phone, email
specialties (JSON array)
sourceCountries (JSON array)
verificationStatus
totalTransactions, successfulTransactions
averageRating
```

5. Inventory Table

Plain Text

```
id (Primary Key)
supplierId (Foreign Key → users)
productType, quantity
condition, grade
sourceCountry
price, currency
status (available/reserved/sold)
```

6. Inspections Table

Plain Text

```
id (Primary Key)
auctionId (Foreign Key → auctions)
bidId (Foreign Key → bids)
```

```
inspectionType (video/physical)
scheduledDate, completedDate
notes, reportUrl
status (pending/scheduled/completed)
```

7. Transactions Table

Plain Text

```
id (Primary Key)
auctionId (Foreign Key → auctions)
buyerId, supplierId (Foreign Keys → users)
totalAmount, currency
paymentStatus
shippingStatus
completedDate
```

8. Ratings Table

Plain Text

```
id (Primary Key)
transactionId (Foreign Key → transactions)
ratedBy, ratedUser (Foreign Keys → users)
rating (1-5)
review, comment
createdAt
```

9. Subscriptions Table

Plain Text

```
id (Primary Key)
userId (Foreign Key → users)
tier (on-demand/starter/growth/enterprise)
monthlyFee, currency
features (JSON array)
startDate, endDate
status (active/inactive)
```

API Endpoints (tRPC Procedures)

Auctions Router

TypeScript

```
// List all active auctions
GET /api/trpc/auctions.list
Query: { limit: 20, offset: 0 }
Response: Auction[]

// Get auction details
GET /api/trpc/auctions.getById
Query: { id: number }
Response: Auction

// Create new auction (buyer only)
POST /api/trpc/auctions.create
Body: {
  title: string,
  description: string,
  productType: string,
  quantity: number,
  condition: 'new' | 'like-new' | 'good' | 'fair',
  startingPrice: number,
  reservePrice?: number,
  endDate: Date,
  deliveryLocation: string,
  paymentTerms?: string
}
Response: { id: number, status: 'draft' }

// Close auction
POST /api/trpc/auctions.close
Body: { auctionId: number }
Response: { success: boolean }

// Update auction
POST /api/trpc/auctions.update
Body: { id: number, ...updates }
Response: Auction

// Delete auction
POST /api/trpc/auctions.delete
Body: { id: number }
Response: { success: boolean }
```

Bids Router

TypeScript

```
// Place a bid
POST /api/trpc/bids.place
Body: {
  auctionId: number,
  bidPrice: number,
  quantity: number,
  paymentTerms?: string,
  notes?: string,
  estimatedDelivery?: Date
}
Response: { id: number, status: 'active' }

// Get my bids
GET /api/trpc/bids.myBids
Response: Bid[]

// Withdraw bid
POST /api/trpc/bids.withdraw
Body: { bidId: number }
Response: { success: boolean }

// Accept bid (buyer only)
POST /api/trpc/bids.accept
Body: { bidId: number }
Response: { success: boolean, transactionId: number }

// Reject bid (buyer only)
POST /api/trpc/bids.reject
Body: { bidId: number, reason?: string }
Response: { success: boolean }

// Get auction bids
GET /api/trpc/bids.getAuctionBids
Query: { auctionId: number }
Response: Bid[]
```

Suppliers Router

TypeScript

```
// Get supplier profile
GET /api/trpc/suppliers.profile
Response: Supplier

// Update supplier profile
```

```
POST /api/trpc/suppliers.updateProfile
```

```
Body: {  
  companyName: string,  
  contactPerson: string,  
  phone: string,  
  email: string,  
  specialties: string[],  
  sourceCountries: string[]  
}
```

```
Response: Supplier
```

```
// Get supplier ratings
```

```
GET /api/trpc/suppliers.getRatings
```

```
Query: { supplierId: number }
```

```
Response: Rating[]
```

```
// Verify supplier (admin only)
```

```
POST /api/trpc/suppliers.verify
```

```
Body: { supplierId: number, status: 'verified' | 'rejected' }
```

```
Response: { success: boolean }
```

Transactions Router

TypeScript

```
// Get transaction details
```

```
GET /api/trpc/transactions.getById
```

```
Query: { id: number }
```

```
Response: Transaction
```

```
// Get my transactions
```

```
GET /api/trpc/transactions.list
```

```
Response: Transaction[]
```

```
// Update transaction status
```

```
POST /api/trpc/transactions.updateStatus
```

```
Body: { id: number, status: 'pending' | 'completed' | 'failed' }
```

```
Response: Transaction
```

Ratings Router

TypeScript

```
// Create rating
```

```
POST /api/trpc/ratings.create
```

```
Body: {
```



```
transactionId: number,  
ratedUser: number,  
rating: 1 | 2 | 3 | 4 | 5,  
review: string  
}  
Response: Rating  
  
// Get ratings for user  
GET /api/trpc/ratings.getForUser  
Query: { userId: number }  
Response: Rating[]
```

Environment Variables Required

Create a `.env` file with these variables:

Plain Text

```
# Database  
DATABASE_URL=mysql://username:password@host:port/database  
  
# Authentication  
JWT_SECRET=your-super-secret-jwt-key-min-32-chars  
VITE_APP_ID=your-oauth-app-id  
OAUTH_SERVER_URL=https://api.manus.im  
VITE_OAUTH_PORTAL_URL=https://portal.manus.im  
  
# Owner Info  
OWNER_OPEN_ID=your-owner-open-id  
OWNER_NAME=Your Company Name  
  
# App Configuration  
VITE_APP_TITLE=ChainTrack  
VITE_APP_LOGO=https://your-domain.com/logo.png  
  
# API Keys  
BUILT_IN_FORGE_API_URL=https://api.manus.im  
BUILT_IN_FORGE_API_KEY=your-api-key  
  
# Optional: Analytics  
VITE_ANALYTICS_ENDPOINT=https://analytics.example.com  
VITE_ANALYTICS_WEBSITE_ID=your-website-id
```

Installation Steps

Step 1: Create Replit Project

1. Go to replit.com
2. Click "Create"
3. Select "Import from GitHub" or "Create from template"
4. Choose **Node.js**
5. Name: `chaintrack-reverse-bidding`

Step 2: Clone Repository

Bash

```
git clone <your-github-repo> .
```

Or upload files manually via Replit's file uploader.

Step 3: Install Dependencies

Bash

```
pnpm install
```

Step 4: Configure Environment

Click the **Secrets** icon and add all environment variables from `.env` template above.

Step 5: Initialize Database

Bash

```
pnpm db:push
```

Step 6: Start Server

Bash

```
pnpm dev
```

Step 7: Access Application

- **Frontend:** `https://<project>.replit.dev`
- **API:** `https://<project>.replit.dev/api/trpc`
- **WebSocket:** `wss://<project>.replit.dev`

File Structure

Plain Text

```
chaintrack_reverse_bidding/
|
├── client/                                # React Frontend
│   ├── src/
│   │   ├── pages/
│   │   │   ├── Home.tsx                 # Landing page
│   │   │   ├── AuctionsList.tsx         # Browse auctions
│   │   │   ├── AuctionDetail.tsx        # Auction details
│   │   │   ├── SupplierDashboard.tsx
│   │   │   ├── MyBids.tsx               # Bid tracking
│   │   │   └── SupplierProfile.tsx      # Profile management
│   │   ├── components/
│   │   │   ├── SupplierLayout.tsx       # Navigation layout
│   │   │   ├── PlaceBidDialog.tsx       # Bid placement modal
│   │   │   └── ui/                      # shadcn/ui components
│   │   ├── lib/
│   │   │   └── trpc.ts                  # tRPC client setup
│   │   ├── App.tsx                     # Main router
│   │   └── main.tsx                    # React entry point
│   ├── index.html
│   └── package.json
|
├── server/                               # Express Backend
│   ├── routers.ts                      # tRPC procedures
│   ├── db.ts                          # Database queries
│   └── _core/
│       ├── context.ts                  # tRPC context
│       ├── trpc.ts                     # tRPC setup
│       ├── env.ts                      # Environment config
│       └── cookies.ts                  # Session management
```

```
├── llm.ts # LLM integration
├── imageGeneration.ts # Image generation
├── voiceTranscription.ts # Audio transcription
└── notification.ts # Notifications

├── drizzle/ # Database Schema
│   ├── schema.ts # Table definitions
│   └── migrations/ # Database migrations
├── storage/ # S3 File Storage
│   └── index.ts # Storage helpers
├── shared/ # Shared Code
│   ├── const.ts # Constants
│   └── types.ts # TypeScript types
├── package.json # Dependencies
├── tsconfig.json # TypeScript config
├── vite.config.ts # Vite config
└── .env.example # Environment template
```

Key Technologies

Technology	Purpose	Version
React	Frontend UI library	19
Express	Backend server framework	4
tRPC	Type-safe RPC framework	11
Drizzle ORM	Database ORM	Latest
MySQL/TiDB	Database	Latest
TypeScript	Type safety	5
Tailwind CSS	Styling	4
Vite	Build tool	Latest
Node.js	Runtime	18+

Common Commands

Bash

```
# Install dependencies
pnpm install

# Push database schema
pnpm db:push

# Start development server
pnpm dev

# Build for production
pnpm build

# Start production server
pnpm start

# Run type checking
pnpm type-check

# Format code
pnpm format

# Lint code
pnpm lint
```

Troubleshooting

Database Connection Failed

Bash

```
# Verify DATABASE_URL
echo $DATABASE_URL

# Test connection
pnpm db:push --verbose
```

Port 3000 Already in Use

Bash

```
# Use different port
PORT=3001 pnpm dev
```

Dependencies Not Installing

Bash

```
# Clear cache
rm -rf node_modules pnpm-lock.yaml
pnpm install
```

OAuth Login Not Working

- Verify `VITE_APP_ID` is correct
- Check `OAUTH_SERVER_URL` is accessible
- Whitelist your Replit URL in OAuth settings

Deployment Checklist

- ☐ Environment variables configured
- ☐ Database schema pushed
- ☐ Dependencies installed
- ☐ Development server running
- ☐ OAuth login working
- ☐ Supplier dashboard accessible
- ☐ Auction creation working
- ☐ Bidding functionality tested
- ☐ Profile management working
- ☐ Database backups configured
- ☐ HTTPS enabled
- ☐ Rate limiting configured
- ☐ Error logging enabled
- ☐ Analytics integrated

- ☐ Payment processing tested
-

Performance Tips

1. **Enable Query Caching** - Cache auction lists for 5 minutes
 2. **Database Indexing** - Index frequently queried columns
 3. **Image Optimization** - Use S3 for image storage
 4. **API Rate Limiting** - Prevent abuse
 5. **Lazy Loading** - Load components on demand
 6. **CDN** - Serve static assets from CDN
-

Security Checklist

- ☐ All environment variables are secrets (not in code)
 - ☐ HTTPS enabled in production
 - ☐ CORS properly configured
 - ☐ Input validation on all endpoints
 - ☐ SQL injection protection (using Drizzle ORM)
 - ☐ XSS protection enabled
 - ☐ Rate limiting implemented
 - ☐ Authentication required for sensitive endpoints
 - ☐ Admin endpoints protected
 - ☐ Database backups configured
-

Next Steps

1. **Customize Branding** - Update logo and colors
2. **Add Payment Gateway** - Integrate Stripe/PayPal
3. **Email Notifications** - Set up SendGrid
4. **SMS Alerts** - Add Twilio integration
5. **Mobile App** - Build React Native version

6. **Admin Dashboard** - Create moderation panel
 7. **Analytics** - Integrate Mixpanel/Amplitude
 8. **Live Chat** - Add customer support
-

Support

For issues and questions:

-  Email: support@chaintrack.com
 -  Discord: [Join Community](#)
 -  Docs: [Documentation](#)
 -  Issues: [GitHub Issues](#)
-

License

Proprietary - All Rights Reserved © 2024 ChainTrack